Mawlana Bhashani Science and Technology University

Department of Information and Communication Technology (ICT)

TCP Congestion and Slow Start Algorithm

How TCP finds a safe sending rate without overloading the network

Monira Akter Mahin

ID: IT24017

Department of ICT, Mawlana Bhashani Science and Technology University, Bangladesh

This presentation explains the basic mechanism TCP uses to control data transmission and maintain network stability. It walks through slow start, congestion avoidance, fast retransmit and fast recovery, and compares how TCP Tahoe and Reno respond to packet loss.

Session 2023-24

Santosh, Tangail, Bangladesh

Presentation Outline

01 Introduction and research problem

Why congestion happens and which question this presentation answers.

02 Related work and research gap

How Tahoe, Reno, CUBIC and BBR differ, and what a simple explanation still lacks.

03 Proposed solution and methodology

The Reno process: slow start, congestion avoidance and fast recovery.

04 Experiments and results

An idealised comparison of how Tahoe and Reno recover after the same loss.

05 Conclusion

The main lessons, the limits of the model and ideas for real measurements.

We will first understand the congestion problem, then explore TCP control phases, compare algorithms, and discuss the results. Each section builds on the previous one, following the life of a single TCP connection.

Introduction and Research Problem

How can a TCP sender use the network fully without causing congestion?

Why this problem matters

01 Overloaded links delay and drop packets, and throughput can collapse for everyone. When many senders react by retransmitting at the same time, the congestion gets worse instead of better.

What current approaches leave unresolved

02 TCP cannot see the network, so it must probe the capacity and react to loss. Because the sender has no direct view of router queues, it has to infer congestion from delayed or missing acknowledgements.

Research question or objective

03 Explain how slow start and congestion control adjust the sending rate. The aim is to show clearly how the congestion window changes after each acknowledgement, timeout and loss event.

The main challenge is to balance high network utilisation with reliable delivery. TCP adjusts its transmission rate based on feedback from acknowledgements and packet loss. This feedback loop keeps the Internet usable when millions of flows share the same links.

Background: What Is Network Congestion?

TCP has to sense congestion by itself and slow down before the network collapses.

1

Too much traffic

Many senders push more data than a router link can carry. The incoming rate simply exceeds the outgoing capacity.

2

Queues overflow

Router buffers fill up, delay rises and packets are dropped. Senders then wait longer for acknowledgements.

3

Throughput falls

Retransmissions add load, so useful throughput drops (congestion collapse). Much capacity is wasted.

Congestion is similar to a traffic jam on a road. When packets arrive faster than a router can forward them, they wait in buffers, causing delay; when buffers become full, packets are discarded. TCP therefore treats missing acknowledgements as a signal to slow down.

Related Work and Research Gap

Study / approach	Key idea	Relevant limitation
TCP Tahoe (Jacobson, 1988)	Slow start, congestion avoidance and fast retransmit, the first fix for collapse.	Any loss sends cwnd back to 1 segment. Even mild loss is costly.
TCP Reno / NewReno	Adds fast recovery to keep the pipe full after a loss, avoiding a full restart.	Reacts slowly to several losses in one window. A timeout may follow.
CUBIC and BBR	Cubic window growth; rate set from bandwidth and RTT estimates for fast links.	Fairness issues between algorithms that share a link.

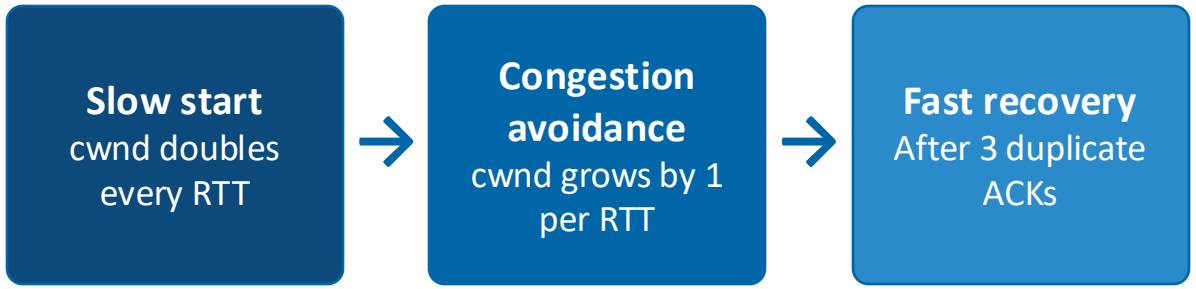
Research gap

A simple step-by-step view of how cwnd changes in each TCP phase. Few short explanations follow one connection from start to recovery.

Tahoe, Reno, CUBIC, and BBR represent different approaches to congestion control. This presentation focuses on the Reno process and uses a simple comparison to make the changes in the congestion window easier to understand.

Methodology

TCP congestion control (Reno)



- cwnd: congestion window, the segments allowed in flight. It caps unacknowledged data.
- ssthresh: slow start threshold, where growth turns linear. It drops after each loss.
- Timeout: ssthresh = cwnd / 2 and cwnd = 1 segment

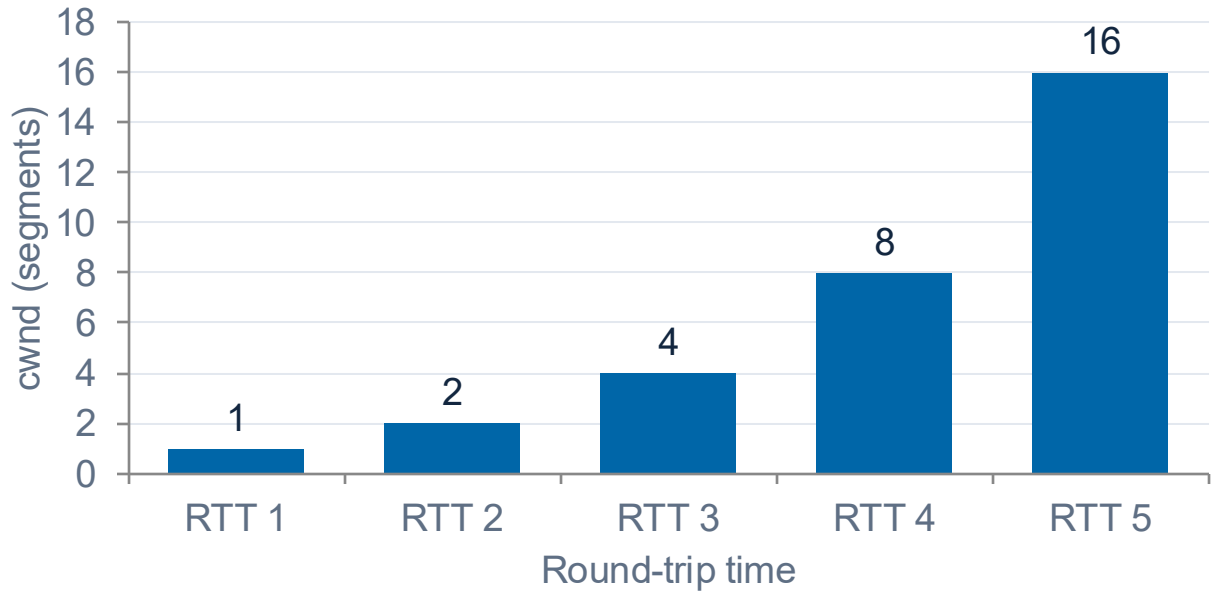
01 **Step 1: Initialise**
Start with a small cwnd and a large ssthresh. The path is unknown, so TCP begins cautiously.

02 **Step 2: Probe and react**
Grow cwnd on every ACK and watch for loss. Duplicate ACKs or a timeout signal congestion.

03 **Step 3: Adapt**
Set ssthresh to cwnd / 2 and resume growth. TCP then probes more slowly near the old limit.

The congestion window (cwnd) limits how much unacknowledged data a sender can place in the network. The sender begins cautiously, increases its rate while acknowledgements arrive, and reduces the rate when congestion is detected.

Slow Start Algorithm



Idealised example with an initial window of 1 segment. Each bar shows cwnd after one round trip.

01 Start small

cwnd begins at a small initial window, then grows with every ACK. This cautious start avoids flooding an unknown path.

02 Double each RTT

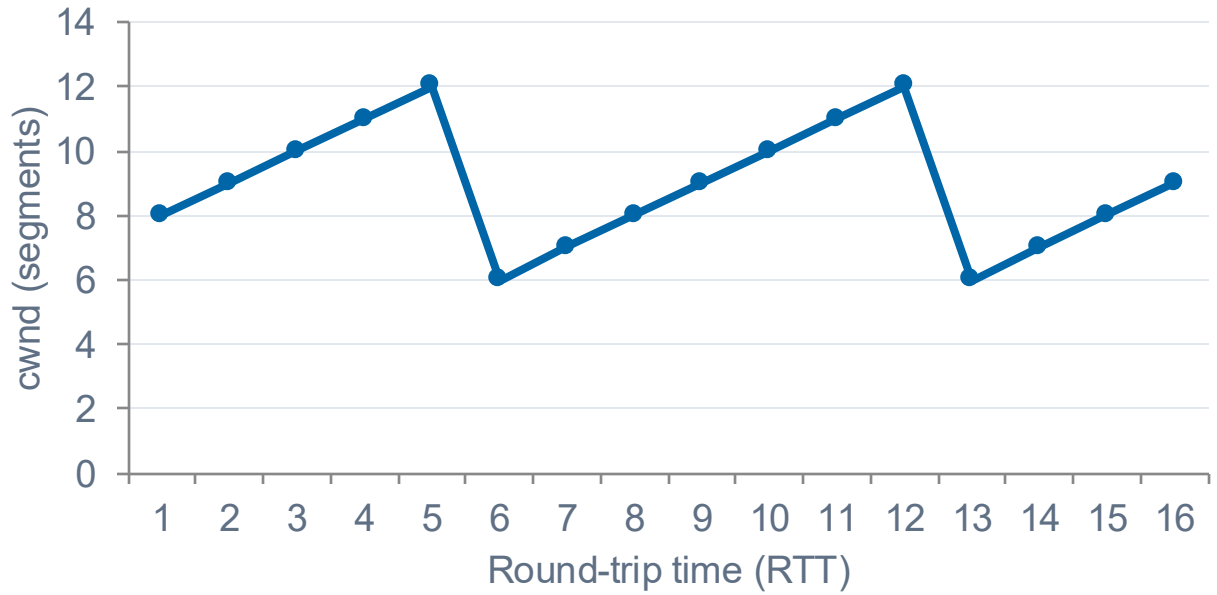
Each ACK adds 1 segment, so cwnd doubles every round trip. The growth is therefore exponential.

03 Stop at ssthresh

At ssthresh, or on loss, TCP leaves slow start. Growth then becomes linear to avoid overshooting.

For example, the congestion window can grow from 1 to 2, then 4, 8, and 16 segments over successive RTTs. This rapid growth helps TCP discover available capacity, but it must stop or change phase when the threshold is reached or loss occurs.

Congestion Avoidance and AIMD



Idealised sawtooth: +1 segment per RTT, halved on each loss. The pattern repeats while the connection keeps probing.

01 Additive increase

cwnd grows by 1 segment per RTT to probe for spare capacity. The slow, steady climb avoids overloading the link.

02 Multiplicative decrease

On loss, cwnd is cut to half so queues can drain. The sharp cut quickly relieves the congested router.

03 Fair sharing

The sawtooth lets competing flows converge to a fair share. Flows with larger windows lose more, so shares even out.

AIMD means Additive Increase and Multiplicative Decrease. TCP increases the window carefully during normal operation and makes a larger reduction after detecting loss, helping prevent persistent congestion.

Fast Retransmit and Fast Recovery



Third duplicate ACK: the sender resends segment 2 at once, without waiting for the timer. This saves a full timeout delay.

3 duplicate ACKs: mild congestion

Fast retransmit, then fast recovery: ssthresh = cwnd / 2 and cwnd continues from there. Only one segment was lost, so data is still flowing.

Timeout: severe congestion

ssthresh = cwnd / 2, cwnd = 1 segment, and TCP restarts in slow start. No ACKs arrive, so the sender must rebuild its rate slowly.

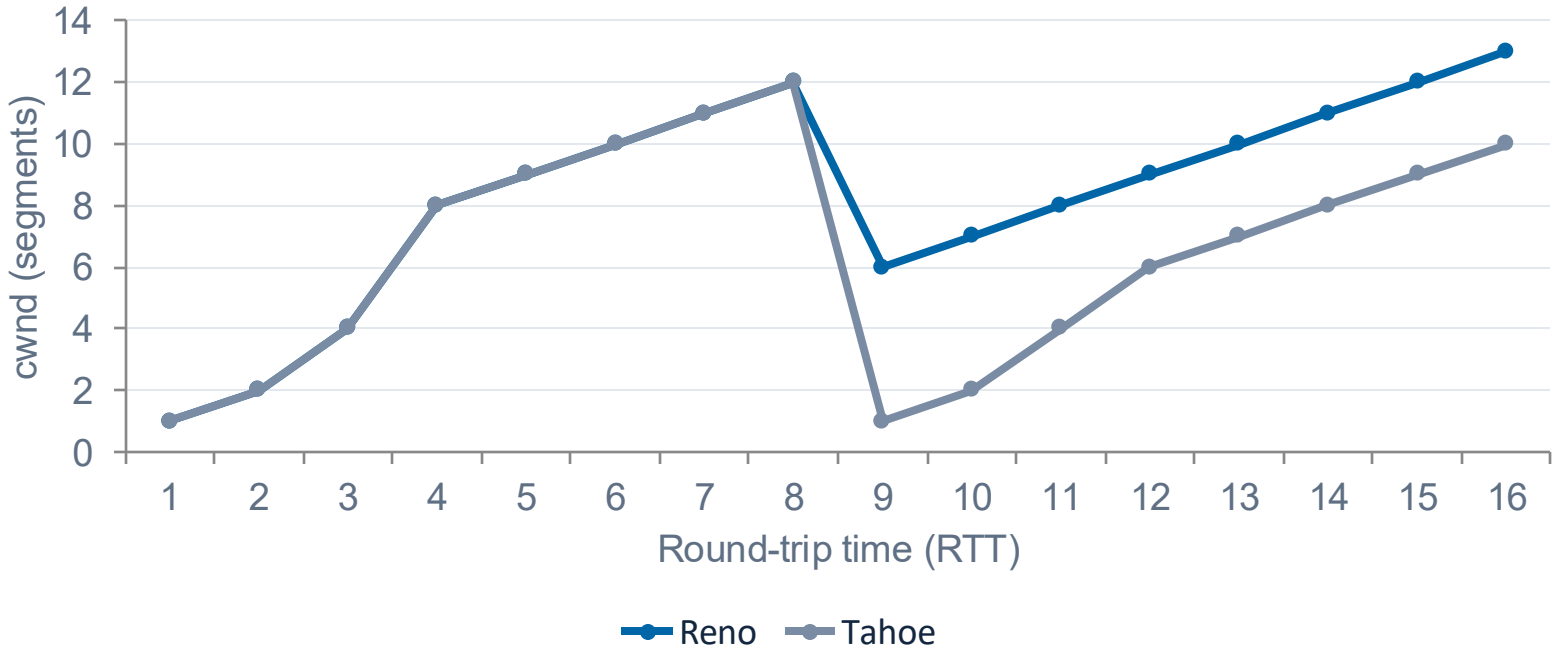
Duplicate acknowledgements indicate that later segments have arrived while an earlier segment is missing. Receiving three duplicate ACKs gives the sender evidence that it can retransmit the missing segment immediately instead of waiting for a timeout.

Experimental Setup

Data / study setting	Idealised single TCP flow over one bottleneck link (no dataset). The sender always has data to send and the link capacity stays fixed.
Baselines / comparisons	TCP Tahoe and TCP Reno, with the same ssthresh and the same loss. Only the reaction to three duplicate ACKs differs.
Evaluation metrics	cwnd in segments per RTT, and recovery time after the loss. Recovery time counts the RTTs needed to regain the old window.
Implementation / protocol	Textbook RTT-round model; ns-3 or Wireshark can reproduce it. Each step of the model equals one round trip time.

The experiment is an illustrative model rather than a live network measurement. Keeping the bottleneck, threshold, and loss event the same makes it easier to compare how Tahoe and Reno respond to congestion.

Results and Discussion



Main finding

After a loss at RTT 8, Reno restarts from half the window, while Tahoe restarts from 1. Until then, both follow the same path.

So Reno regains its sending rate much sooner. Tahoe gives up that throughput.

Figure: idealised cwnd (segments) per RTT, initial ssthresh = 8, 3 duplicate ACKs at RTT 8. Not measured data.

The graph shows the different recovery behaviour after the same loss event. Reno uses fast recovery after three duplicate ACKs, whereas Tahoe returns to a one-segment window and begins slow start again.

Quantitative Comparison

Method	On 3 duplicate ACKs	On timeout	Window growth
TCP Tahoe	cwnd = 1, slow start	cwnd = 1, slow start	+1 segment per RTT after ssthresh
TCP Reno	cwnd halves, fast recovery	cwnd = 1, slow start	+1 segment per RTT
CUBIC	**cwnd cut to 70%**	**cwnd = 1, slow start**	**Cubic function of time since loss**

Fast recovery spares Reno the full restart from one segment that Tahoe suffers. CUBIC cuts cwnd only to 70% and then regrows it along a cubic curve.

The comparison highlights why the response to duplicate ACKs matters. Reno can preserve more of its transmission progress, while a timeout is treated as a stronger sign of congestion by both algorithms.

Conclusions and Future Work

Slow start, AIMD and fast recovery let TCP find and share bandwidth safely.

Primary contribution

01 A step-by-step view of how cwnd evolves in each TCP phase. It links slow start, AIMD, fast retransmit and fast recovery into one clear sequence.

Important limitation

02 An idealised model: real networks add noise, buffers and competing flows. The curves show the logic of each algorithm, not measured performance.

Next research step

03 Measure real flows with Wireshark or ns-3 and compare Reno, CUBIC and BBR. Such tests would show how each algorithm copes with different delays and loss rates.

Overall, TCP uses feedback to adapt its sending rate instead of transmitting at a fixed speed. Future testing in a simulator or packet-analysis tool can show how these ideas behave under different network conditions.

Selected References

[1] V. Jacobson, “Congestion avoidance and control,” ACM SIGCOMM, 1988.
The original paper that introduced slow start and congestion avoidance for TCP.

[2] M. Allman, V. Paxson and E. Blanton, “TCP congestion control,” RFC 5681, IETF, 2009.
The standard that defines slow start, congestion avoidance, fast retransmit and fast recovery.

[3] L. Xu, S. Ha, I. Rhee, V. Goel and L. Eggert, “CUBIC for fast and long-distance networks,” RFC 9438, IETF, 2023.
Describes CUBIC, a congestion control algorithm widely used on today’s Internet.

[4] N. Cardwell et al., “BBR: Congestion-based congestion control,” Communications of the ACM, 2017.
Introduces BBR, which sets the sending rate from bandwidth and round-trip time estimates.

These references provide the theoretical foundation for TCP congestion control and the algorithms discussed in this presentation. They can be consulted for protocol definitions, design details, and further study.

Thank you

Questions and Discussion

Monira Akter Mahin
Department of ICT, Mawlana Bhashani Science and Technology University
ID: IT24017 | Session 2023-24

I would be happy to discuss any part of the process, from slow start to fast recovery.

Thank you for your attention. I hope this presentation has clarified how TCP detects congestion, adjusts its congestion window, and recovers from packet loss.